Python Programming

Basics & Fundamentals

Course Documentation

Includes: Video Resources | Module Quizzes | Long-Answer Practice

Beginner Level | Self-Paced

Table of Contents

1. Module 1: Introduction to Python
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
2. Module 2: Variables & Data Types
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
3. Module 3: Operators
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
4. Module 4: Control Flow
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
5. Module 5: Functions
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
6. Module 6: Lists, Tuples & Dictionaries
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
7. Module 7: File Handling
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A
8. Module 8: Error Handling
   📺 Video Resources 📝 Quiz 📖 Long-Answer Q&A

Module 1: Introduction to Python

📝 Blog Introduction

Welcome to the world of Python programming! This module lays the foundation for your coding journey. You'll discover what Python is, why it's one of the most popular and beginner-friendly programming languages in the world, and how to set up your development environment. By the end of this module, you'll write and run your very first Python program — a milestone every programmer remembers.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Tutorial for Beginners – Full Course *Complete beginner Python course by freeCodeCamp*	4 hrs 26 min	**YouTube**
▶️ Python in 100 Seconds *Quick overview of Python by Fireship*	2 min	**YouTube**
▶️ How to Install Python & Set Up VS Code *Step-by-step environment setup guide*	12 min	**YouTube**

📝 Module Quiz

Q1. Who created the Python programming language?

A) Dennis Ritchie
B) Guido van Rossum
C) James Gosling
D) Bjarne Stroustrup

✅ **Answer: B) Guido van Rossum**
Explanation: Python was created by Guido van Rossum and first released in 1991.

Q2. Which command is used to display output in Python?

A) echo()
B) console.log()
C) print()
D) output()

✅ **Answer: C) print()**
Explanation: The print() function is the standard way to display output in Python.

Q3. What symbol is used to write comments in Python?

A) //
B) /* */
C) --
D) #

✅ **Answer: D) #**
Explanation: Python uses the # symbol. Everything after # on the same line is ignored by the interpreter.

Q4. Python is a/an _____ language.

A) Compiled
B) Interpreted
C) Assembly
D) Machine-level

✅ **Answer: B) Interpreted**
Explanation: Python is an interpreted language — code is executed line by line at runtime.

📖 Long-Answer Practice Questions

1. What is Python and what are its main characteristics that make it suitable for beginners?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991. It is designed with a philosophy that emphasizes code readability and simplicity.

Key characteristics that make Python beginner-friendly include:

• Simple syntax: Python's syntax is clean and reads almost like English, reducing the learning curve significantly compared to languages like C++ or Java.

• Dynamic typing: Variables do not need type declarations; Python infers types automatically.

• Interpreted execution: Code runs line by line, making it easy to test and debug interactively.

• Huge standard library: Python includes thousands of built-in modules covering everything from file I/O to networking.

• Cross-platform: Python runs on Windows, macOS, Linux, and many other platforms without modification.

• Strong community: Python has one of the largest and most supportive programming communities worldwide.

2. Explain the steps required to install Python and verify that the installation was successful.

Follow these steps to install Python on your machine:

Step 1 — Download: Visit the official website python.org/downloads and download the latest stable version (Python 3.x). Always prefer the latest 3.x version over Python 2, which is no longer supported.

Step 2 — Install: Run the downloaded installer. On Windows, make sure to check the box that says "Add Python to PATH" before clicking Install. This ensures Python can be run from any terminal window.

Step 3 — Verify: Open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type: python --version or python3 --version. You should see output like: Python 3.11.4.

Step 4 — Test the interpreter: Type python in the terminal to open the interactive shell (REPL). Type print("Hello") and press Enter. If you see Hello, the installation is working correctly.

Step 5 — Install an IDE: Download VS Code or PyCharm for a better development experience. Install the Python extension in VS Code for syntax highlighting and debugging support.

3. Write a Python program that takes a user's name as input and prints a personalized greeting. Explain each line.

```
# Program to greet the user
name = input("What is your name? ")   # Prompts user and stores input in "name"
print(f"Hello, {name}! Welcome to Python.")  # Prints personalized greeting
```

Explanation of each line:

• Line 1: A comment (starts with #) — explains what the program does. It is ignored by Python.

• Line 2: input() displays a prompt message and waits for the user to type something. The entered text is stored as a string in the variable name.

• Line 3: print() displays the output. The f before the quotes makes it an f-string, which allows the variable {name} to be inserted directly into the text.

Module 2: Variables & Data Types

📝 Blog Introduction

In this module, you'll explore the building blocks of every Python program: variables and data types. Learn how Python stores and manages information using strings, integers, floats, and booleans. Understanding how to declare variables, assign values, and work with different data types is essential for writing meaningful, dynamic programs.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Variables and Data Types – Explained *In-depth explanation of variables, types and type conversion*	18 min	**YouTube**
▶️ Python Strings – Full Tutorial *All string operations, methods and f-strings by Corey Schafer*	30 min	**YouTube**
▶️ Python Type Casting Explained *How to convert between int, float, str and bool*	10 min	**YouTube**

📝 Module Quiz

Q1. Which of the following is a valid variable name in Python?

A) 2name
B) my-name
C) my_name
D) my name

✅ **Answer: C) my_name**
Explanation: Variable names cannot start with a digit, contain hyphens, or include spaces. Underscores are allowed.

Q2. What is the data type of the value: 3.14?

A) int
B) str
C) float
D) bool

✅ **Answer: C) float**
Explanation: Any number with a decimal point is a float in Python.

Q3. What does int('42') return?

A) '42'
B) 42.0
C) 42
D) Error

✅ **Answer: C) 42**
Explanation: int() converts a string or float to an integer. '42' becomes the integer 42.

Q4. Which function returns the data type of a variable?

A) datatype()
B) typeof()
C) type()
D) kind()

✅ **Answer: C) type()**
Explanation: type(variable) returns the class/type of the given variable.

📖 Long-Answer Practice Questions

1. Explain the five core data types in Python with examples for each.

Python has five primary built-in data types:

1. int (Integer): Whole numbers without a decimal point. Examples: 0, -5, 100, 999. Used for counting, indexing, loop counters, etc. Example: age = 25

2. float (Floating-point): Numbers with a decimal point. Examples: 3.14, -0.5, 100.0. Used for measurements, calculations requiring precision. Example: temperature = 98.6

3. str (String): A sequence of characters enclosed in single or double quotes. Examples: "Hello", 'Python', "123". Strings are immutable (cannot be changed in-place). Example: name = "Alice"

4. bool (Boolean): Represents logical values — either True or False (capital first letter). Used in conditions, comparisons, and control flow. Example: is_logged_in = True

5. NoneType: Represents the absence of a value. The only value of NoneType is None. Used as a default or placeholder. Example: result = None

2. What are f-strings in Python and why are they preferred over older string formatting methods?

f-strings (formatted string literals) were introduced in Python 3.6. They allow you to embed expressions inside string literals using curly braces {}.

Syntax: f"text {variable_or_expression} more text"

Example: name = "Alice"; age = 25; print(f"My name is {name} and I am {age} years old.")

Why f-strings are preferred:

• Readability: The variable is written directly where it appears in the string, making it easy to understand at a glance.

• Conciseness: Less code to write compared to older methods like % formatting or .format().

• Performance: f-strings are faster than .format() and % formatting at runtime.

• Expression support: You can include any valid Python expression inside {}, for example: f'Result: {2 + 3}' or f'Upper: {name.upper()}'.

Comparison: Old way: "Hello, %s" % name or "Hello, {}".format(name) vs New way: f"Hello, {name}"

Module 3: Operators

📝 Blog Introduction

Operators are the tools Python uses to perform computations and make decisions. In this module, you'll master arithmetic, comparison, logical, and assignment operators. From simple addition to complex logical expressions, operators power the logic behind every program you'll ever write.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Operators – All Types Explained *Arithmetic, comparison, logical, bitwise and assignment operators*	22 min	**YouTube**
▶️ Python Comparison & Logical Operators *True/False logic, and/or/not with real-world examples*	14 min	**YouTube**

📝 Module Quiz

Q1. What is the result of 10 % 3 in Python?

A) 3
B) 1
C) 0
D) 3.33

✅ **Answer: B) 1**
Explanation: The % operator returns the remainder of division. 10 ÷ 3 = 3 remainder 1.

Q2. Which operator checks if two values are equal?

A) =
B) ==
C) ===
D) :=

✅ **Answer: B) ==**
Explanation: == is the equality comparison operator. A single = is the assignment operator.

Q3. What does 'True and False' evaluate to?

A) True
B) False

C) None
D) Error

✅ **Answer: B) False**
Explanation: The 'and' operator returns True only if both operands are True. Since one is False, the result is False.

Q4. What is the result of 2 ** 3?

A) 6
B) 5
C) 8
D) 9

✅ **Answer: C) 8**
*Explanation: The ** operator means exponentiation. 2 ** 3 means 2 to the power of 3, which equals 8.*

📖 Long-Answer Practice Questions

1. Explain the difference between the / (division) and // (floor division) operators in Python with examples.

Python provides two different division operators:

/ (True Division): Always returns a float, even when the result is a whole number.

Examples: 10 / 2 → 5.0, 7 / 2 → 3.5, 10 / 3 → 3.3333...

// (Floor Division): Returns the largest whole number (integer) that is less than or equal to the result. It 'floors' the result — it rounds down.

Examples: 10 // 2 → 5, 7 // 2 → 3, 10 // 3 → 3

Important notes:

• Floor division always rounds toward negative infinity, not toward zero. So -7 // 2 gives -4 (not -3).
• Use / when you need a precise decimal result (e.g., splitting a bill)
• Use // when you need a whole number result (e.g., pagination, grouping items)

2. What are logical operators in Python? Describe 'and', 'or', and 'not' with truth tables and examples.

Logical operators are used to combine or modify boolean (True/False) expressions. Python has three: and, or, not.

AND operator — returns True only if BOTH conditions are True:

True and True → True
True and False → False
False and True → False
False and False → False
Example: age = 20; is_member = True; if age >= 18 and is_member: print('Access granted')

OR operator — returns True if AT LEAST ONE condition is True:

True or False → True
False or True → True
False or False → False

Example: has_ticket = False; is_vip = True; if has_ticket or is_vip: print('Entry allowed')

NOT operator — reverses the boolean value:

not True → False

not False → True

Example: is_locked = False; if not is_locked: print('Door is open')

Module 4: Control Flow

📝 Blog Introduction

Control flow determines the order in which your code executes — and this module puts you in the driver's seat. You'll learn to use if/elif/else statements to make decisions, and loops (for and while) to repeat actions. These structures are the backbone of intelligent, responsive programs.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python If Statements – Complete Tutorial *if, elif, else with real-world scenarios by Corey Schafer*	20 min	**YouTube**
▶️ Python For Loops and While Loops *Loops with range(), break, continue and nested loops*	25 min	**YouTube**
▶️ Python Loops Deep Dive *Advanced loop patterns, comprehensions preview*	18 min	**YouTube**

📝 Module Quiz

Q1. What will this code print? for i in range(3): print(i)

A) 1 2 3
B) 0 1 2
C) 0 1 2 3
D) 1 2

✅ **Answer: B) 0 1 2**
Explanation: range(3) generates 0, 1, 2. It starts at 0 and goes up to (but not including) 3.

Q2. Which keyword is used to skip the current iteration in a loop?

A) skip
B) pass
C) break
D) continue

✅ **Answer: D) continue**
Explanation: continue skips the rest of the current iteration and moves to the next one.

Q3. What indentation does Python require for code blocks?

A) Curly braces {}
B) Consistent spaces or tabs (usually 4 spaces)
C) Semicolons ;
D) Begin/End keywords

✅ **Answer: B) Consistent spaces or tabs (usually 4 spaces)**
Explanation: Python uses indentation to define code blocks. The standard convention is 4 spaces.

Q4. Which statement correctly exits a loop immediately?

A) exit()
B) return
C) break
D) stop

✅ **Answer: C) break**
Explanation: The break statement immediately terminates the innermost loop and continues execution after the loop.

📖 Long-Answer Practice Questions

1. Explain the difference between for loops and while loops in Python. When would you use each one?

Both for and while loops repeat a block of code, but they are suited for different situations:

FOR LOOP: Used when you know how many times to iterate, or when you want to iterate over a collection (list, string, range, etc.).

Syntax: for variable in sequence:

Example: for i in range(5): print(i) → prints 0 to 4

Best for: iterating over lists, strings, dictionaries; running something a fixed number of times

WHILE LOOP: Used when you want to keep looping as long as a condition remains True. The number of iterations is not known in advance.

Syntax: while condition:

Example: while user_input != "quit": user_input = input("Enter command: ")

Best for: waiting for user input, reading until end of file, game loops, polling

Key difference: A for loop has a defined endpoint; a while loop continues until its condition becomes False. An infinite while loop (while True) with a break statement is common for menu-driven programs.

2. Write a Python program using nested loops to print a multiplication table (1 to 5). Explain how nested loops work.

A nested loop is a loop inside another loop. The inner loop runs completely for each iteration of the outer loop.

Code:

```
for i in range(1, 6):          # outer loop: rows 1–5
    for j in range(1, 6):      # inner loop: columns 1–5
        print(i * j, end='\t')  # print product with tab
    print()                    # newline after each row
```

How it works:

- Outer loop iteration 1 (i=1): inner loop runs for j=1,2,3,4,5, printing: 1 2 3 4 5
- Outer loop iteration 2 (i=2): inner loop runs again, printing: 2 4 6 8 10
- This continues until i=5, completing the full 5x5 multiplication table.

The end='\t' argument in print() prevents the default newline and instead adds a tab, keeping numbers on the same row. The print() with no arguments after the inner loop moves to the next line.

Module 5: Functions

📝 Blog Introduction

Functions are the key to writing clean, reusable, and organized code. In this module, you'll learn how to define and call functions, pass arguments, return values, and understand variable scope. Mastering functions transforms you from a script writer into a true programmer who thinks in modular, efficient solutions.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Functions – Complete Beginner Guide *def, parameters, return, scope – everything about functions*	28 min	**YouTube**
▶️ Lambda Functions in Python *When and how to use lambda / anonymous functions*	12 min	**YouTube**
▶️ Python Scope – LEGB Rule Explained *Local, Enclosing, Global, Built-in scope with examples*	20 min	**YouTube**

📝 Module Quiz

Q1. What keyword is used to define a function in Python?

A) function
B) func
C) def
D) define

✅ **Answer: C) def**
Explanation: Python uses the 'def' keyword followed by the function name and parentheses to define a function.

Q2. What does the return statement do?

A) Prints the result
B) Ends the program
C) Exits the function and optionally sends a value back
D) Calls the function again

✅ **Answer: C) Exits the function and optionally sends a value back**
Explanation: return ends the function's execution and can pass a value back to the caller. Without

return, the function returns None.

Q3. Which of the following defines a function with a default parameter?

A) def greet(name = 'World'):
B) def greet(name:'World'):
C) def greet[name='World']:
D) def greet{name='World'}:

✅ **Answer: A) def greet(name = 'World'):**
Explanation: Default parameters are defined using = inside the parentheses.

Q4. What is a lambda function?

A) A function with many parameters
B) A function that never returns
C) A small, anonymous one-line function
D) A function defined inside a class

✅ **Answer: C) A small, anonymous one-line function**
Explanation: Lambda functions are concise, anonymous functions defined using the lambda keyword on a single line.

📖 Long-Answer Practice Questions

1. What is the difference between parameters and arguments in Python functions? Provide examples.

These two terms are often confused but refer to different things:
PARAMETERS are the variable names defined in the function signature (the definition). They act as placeholders.
Example: def add(a, b): → here a and b are parameters
ARGUMENTS are the actual values passed to the function when it is called.
Example: result = add(10, 25) → here 10 and 25 are arguments
Types of arguments:
1. Positional arguments: matched by position. add(10, 25) → a=10, b=25
2. Keyword arguments: matched by name. add(b=25, a=10) → same result
3. Default arguments: parameter has a default value used when no argument is passed.
Example: def greet(name='World'): print(f'Hello, {name}!')
greet() → Hello, World! greet('Alice') → Hello, Alice!

2. Explain variable scope in Python. What is the difference between local and global variables?

Scope refers to where in a program a variable can be accessed. Python follows the LEGB rule: Local → Enclosing → Global → Built-in.
LOCAL SCOPE: Variables defined inside a function are local. They only exist within that function and cannot be accessed outside it.

```
def my_func():
    x = 10      # x is local to my_func
```

```
    print(x)    # works fine
print(x)        # NameError — x does not exist here
```

GLOBAL SCOPE: Variables defined at the top level of a script are global. They can be read inside functions.

```
name = 'Alice'   # global
def greet():
    print(name)  # can read global variable
```

MODIFYING globals inside a function: Use the 'global' keyword.

```
count = 0
def increment():
    global count
    count += 1
```

Best practice: Avoid modifying globals inside functions as it makes code harder to debug. Pass values as arguments and return results instead.

Module 6: Lists, Tuples & Dictionaries

📝 Blog Introduction

Python's built-in data structures — lists, tuples, and dictionaries — let you store, organize, and manipulate collections of data with ease. This module covers how to create and work with each structure, access elements, iterate through them, and choose the right one for the right task.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Lists – Complete Tutorial *Creating, slicing, modifying and sorting lists – by Corey Schafer*	32 min	**YouTube**
▶️ Python Dictionaries – In Depth *Keys, values, methods and looping through dictionaries*	26 min	**YouTube**
▶️ Lists vs Tuples vs Sets in Python *When to choose which collection type*	16 min	**YouTube**

📝 Module Quiz

Q1. Which collection type is immutable (cannot be changed after creation)?

A) List
B) Dictionary
C) Set
D) Tuple

✅ **Answer: D) Tuple**
Explanation: Tuples are immutable. Once created, their elements cannot be added, removed, or changed.

Q2. How do you access the last element of a list in Python?

A) list[last]
B) list[-1]
C) list[0]
D) list.last()

✅ **Answer: B) list[-1]**
Explanation: Negative indexing starts from the end. -1 is the last element, -2 is second-to-last, etc.

Q3. What method adds an element to the END of a list?

A) list.add()
B) list.push()
C) list.append()
D) list.insert()

✅ **Answer: C) list.append()**
Explanation: append() adds the element to the end of the list. insert(index, value) adds at a specific position.

Q4. What is the correct syntax to access the value for key 'age' in a dictionary named 'person'?

A) person.age
B) person('age')
C) person[age]
D) person['age']

✅ **Answer: D) person['age']**
Explanation: Dictionary values are accessed using square brackets with the key as a string in quotes.

📖 Long-Answer Practice Questions

1. Compare and contrast lists, tuples, and dictionaries in Python. When would you use each?

These are Python's three most commonly used data structures, each with distinct characteristics:

LISTS — Ordered, mutable, allow duplicates. Created with square brackets [].

fruits = ['apple', 'banana', 'cherry']

Use when: You need an ordered collection that you'll modify (add/remove/change items).

Examples: shopping cart, student scores, playlist

TUPLES — Ordered, immutable, allow duplicates. Created with parentheses ().

coordinates = (10.5, 20.3)

Use when: Data should not change (constant values, function returns, unpacking).

Examples: GPS coordinates, RGB color values, database records

DICTIONARIES — Unordered (Python 3.7+ maintains insertion order), mutable, key-value pairs. Created with curly braces {}.

person = {'name': 'Alice', 'age': 25}

Use when: You need to look up values by a meaningful key rather than an index.

Examples: user profiles, word frequency counts, configuration settings

Quick summary: List = ordered mutable sequence; Tuple = ordered fixed sequence; Dictionary = labeled data lookup

2. Write a Python program that counts the frequency of each word in a sentence using a dictionary.

Here is a complete solution with explanation:

sentence = "the cat sat on the mat the cat"

```
words = sentence.split()      # Split into a list of words
frequency = {}                # Empty dictionary to hold counts
for word in words:
    if word in frequency:       # If word already seen, increment count
        frequency[word] += 1
    else:                       # If new word, set count to 1
        frequency[word] = 1
print(frequency)
# Output: {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1}
```

Explanation:

- sentence.split() breaks the string into a list by whitespace: ['the','cat','sat',...]
- We loop through each word and check if it's already a key in our dictionary.
- If it is, we increment its count by 1. If not, we create the key with value 1.
- The final dictionary maps each unique word to how many times it appeared.

Shortcut using get(): frequency[word] = frequency.get(word, 0) + 1 (does both if/else in one line)

Module 7: File Handling

📝 Blog Introduction

Programs become truly powerful when they can read from and write to files. In this module, you'll learn how to open, read, write, and close files in Python. Whether you're processing data from a text file or saving results to disk, file handling skills are essential for real-world Python development.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python File Handling – Read & Write Files *Open, read, write, append files with the with statement*	24 min	**YouTube**
▶️ Working with CSV Files in Python *Reading and writing CSV files using the csv module*	18 min	**YouTube**

📝 Module Quiz

Q1. Which file mode opens a file for reading only?

A) 'w'
B) 'a'
C) 'r'
D) 'x'

✅ **Answer: C) 'r'**
Explanation: 'r' opens a file for reading. The file must exist. 'w' writes (overwrites), 'a' appends, 'x' creates new.

Q2. What is the advantage of using 'with open(...)' over 'open(...)'?

A) It's faster
B) It automatically closes the file when done
C) It supports more file types
D) It works only with text files

✅ **Answer: B) It automatically closes the file when done**
Explanation: The 'with' statement (context manager) automatically closes the file even if an error occurs, preventing resource leaks.

Q3. Which method reads all lines of a file into a list?

A) file.read()
B) file.readline()
C) file.readlines()
D) file.getlines()

✅ **Answer: C) file.readlines()**
Explanation: readlines() returns a list where each element is a line from the file as a string (including \n).

📖 Long-Answer Practice Questions

1. Explain the different file opening modes in Python ('r', 'w', 'a', 'r+') and when to use each.

Python's open() function accepts a mode argument that controls how the file is opened:

'r' — Read mode (default): Opens the file for reading only. Raises FileNotFoundError if file doesn't exist.

Use when: You only need to read existing data. Example: loading a config file.

'w' — Write mode: Opens for writing. Creates the file if it doesn't exist. OVERWRITES all existing content if it does.

Use when: You want to create a new file or completely replace an existing file's contents.

Warning: Be careful — all previous content is lost!

'a' — Append mode: Opens for writing but adds content at the END of the file. Creates file if it doesn't exist.

Use when: You want to add new entries to a log file or growing dataset without losing old data.

'r+' — Read and Write mode: Opens for both reading and writing. File must exist. Does not truncate (erase) the file.

Use when: You need to read and update parts of an existing file.

Binary variants: 'rb', 'wb', 'ab' — same as above but for binary files (images, PDFs, etc.)

Module 8: Error Handling

📝 Blog Introduction

Even the best programmers write code that encounters errors — the difference is knowing how to handle them gracefully. This module teaches you Python's exception handling system using try, except, else, and finally blocks. You'll learn to anticipate potential failures, write robust code, and provide helpful error messages to users.

📺 Video Resources

📺 Video Title & Description	Duration	Platform
▶️ Python Exception Handling – try/except *try, except, else, finally with real examples by Corey Schafer*	20 min	**YouTube**
▶️ Python Custom Exceptions *How to raise and create your own exception classes*	15 min	**YouTube**

📝 Module Quiz

Q1. Which block always executes, whether or not an exception occurs?

A) except
B) try
C) else
D) finally

✅ **Answer: D) finally**
Explanation: The 'finally' block always runs regardless of whether an exception was raised or not. Used for cleanup (closing files, releasing resources).

Q2. What exception is raised when you try to divide by zero?

A) ValueError
B) ArithmeticError
C) ZeroDivisionError
D) MathError

✅ **Answer: C) ZeroDivisionError**
Explanation: Python raises ZeroDivisionError specifically when attempting to divide any number by zero.

Q3. Which keyword is used to manually raise an exception?

A) throw
B) raise
C) error
D) except

✅ **Answer: B) raise**
Explanation: The 'raise' keyword is used to deliberately trigger an exception. Example: raise ValueError('Invalid input')

Q4. What does the 'else' block do in a try/except structure?

A) Runs if an exception occurs
B) Runs only if NO exception occurs
C) Always runs at the end
D) Catches all exceptions

✅ **Answer: B) Runs only if NO exception occurs**
Explanation: The 'else' block executes only when the 'try' block completes without raising any exception.

📖 Long-Answer Practice Questions

1. Explain the complete try/except/else/finally structure in Python with a practical example.

Python's error handling uses four blocks that work together:
TRY block: Contains code that might raise an exception. Python executes it and watches for errors.
EXCEPT block: Executes if a specific (or any) exception occurs in the try block. Can have multiple except clauses for different exception types.
ELSE block: Executes only if the try block completes without any exception. Rarely used but useful for code that should run only on success.
FINALLY block: Always executes, no matter what. Used for cleanup tasks.
Complete example:

```
try:
    file = open('data.txt', 'r')
    content = file.read()
    number = int(content.strip())
except FileNotFoundError:
    print('Error: File not found!')
except ValueError:
    print('Error: File does not contain a valid number!')
else:
    print(f'Successfully read number: {number}')
finally:
    print('File operation attempted — program continues.')
```

Best practices:
• Always catch specific exceptions rather than bare 'except:'

- Use finally to close files, database connections, or release resources
- Don't suppress exceptions silently — always handle or log them

2. What is the difference between syntax errors and exceptions in Python? Give examples of each.

Python has two broad categories of errors:

SYNTAX ERRORS: Mistakes in the structure of the code that prevent Python from parsing/understanding it. They are detected BEFORE the program runs.

Examples:

- Missing colon: `if x > 5` (should be: `if x > 5:`)
- Wrong indentation: `print('hello')` placed incorrectly
- Mismatched brackets: `my_list = [1, 2, 3)` ([opened,) closed)

Python shows a SyntaxError message with the line number. The program cannot start at all.

EXCEPTIONS (Runtime Errors): Errors that occur WHILE the program is running, even though the syntax is correct.

Examples:

- ValueError: `int('hello')` — valid syntax, but 'hello' can't be converted to int
- IndexError: `lst[10]` when lst only has 3 items
- TypeError: `'hello' + 5` — can't add string and integer
- ZeroDivisionError: `10 / 0`

Exceptions can be caught and handled using try/except. Syntax errors cannot be caught at runtime.

Summary: Fix syntax errors by correcting your code before running it. Handle exceptions using try/except to make your program robust.

Appendix: Additional Resources

Recommended YouTube Channels

📺 Video Title & Description	Duration	Platform
▶️ freeCodeCamp Python Full Course (12 Hours) *The most comprehensive free Python course on YouTube*	12 hr	**YouTube**
▶️ Corey Schafer – Python Tutorials Playlist *In-depth, high-quality Python tutorial series (highly recommended)*	Playlist	**YouTube**
▶️ Tech With Tim – Python Beginner to Advanced *Projects, games and real-world Python applications*	Channel	**YouTube**
▶️ Sentdex – Python Programming Tutorials *Data science, machine learning and Python deep dives*	Channel	**YouTube**
▶️ CS Dojo – Python for Beginners *Clean, easy-to-follow beginner Python and algorithms*	Channel	**YouTube**

Quick Reference: Essential Built-in Functions

Function	Description & Example
print()	`print("Hello") → displays output`
input()	`name = input("Enter name: ")`
len()	`len([1,2,3]) → 3`
type()	`type(3.14) → <class 'float'>`
range()	`range(5) → 0,1,2,3,4`
int(), float(), str()	`Type conversion functions`
max(), min()	`max([3,1,4]) → 4`
sum()	`sum([1,2,3]) → 6`
sorted()	`sorted([3,1,2]) → [1,2,3]`
enumerate()	`enumerate(['a','b']) → (0,'a'),(1,'b')`

— End of Course Documentation —